Phase 2 Design & Specification

3. Gherkin Scripts

3.1 User Registration

```
Scenario: Successful user registration
  Given no account exists with email "user@uni.edu"
  When the user submits name "Ahmed", email "user@uni.edu", and password
"Password123"
  Then the system creates a new user account
  And the password is stored as a bcrypt hash
  And the system returns HTTP 201
Scenario: Registration with duplicate email
  Given an account already exists with email "user@uni.edu"
  When the user submits the registration form
  Then the system returns HTTP 400
  And the response contains "Email already exists"
```

3.2 User Login

```
Scenario: Successful user login
  Given a registered user exists
  When the user submits valid credentials
  Then the system returns HTTP 200
  And returns a JWT access token
  And returns user information
Scenario: Invalid login
  Given a registered user exists
  When the user submits an incorrect password
  Then the system returns HTTP 401
  And the response contains "Invalid credentials"
```

3.3 Category Management

```
Scenario: Admin creates category
  Given the admin is authenticated
  When the admin submits category name "Electronics"
  Then the system creates the category
  And returns HTTP 201
Scenario: Duplicate category name
  Given category "Electronics" already exists
  When the admin submits category "Electronics"
  Then the system returns HTTP 400
```

```
Scenario: Admin updates category
  Given category ID 1 exists
  When the admin updates the category name
  Then the system returns HTTP 200
Scenario: Admin deletes category
  Given category ID 1 exists
  When the admin deletes the category
  Then the system returns HTTP 200
```

3.4 Product Catalog

```
Scenario Outline: Product search and pagination
  Given products exist in the catalog
  When the user requests page <page> with limit <limit> and search "<term>"
  Then the system returns at most <limit> products
  And includes total, page, and limit

Examples:
  | page | limit | term    |
  | 1    | 9     | laptop  |
  | 2    | 9     | mouse   |
  | 1    | 12    | cable   |
Scenario: Filter products by category
  Given products exist in multiple categories
  When the user requests products with category_id 3
  Then only products in category 3 are returned
Scenario: Product details not found
  Given product ID 999 does not exist
  When the user requests product details
  Then the system returns HTTP 404
```

3.5 Shopping Cart

```
Scenario: Add product to cart
  Given the user is authenticated
  And product stock is greater than zero
  When the user adds 2 units to the cart
  Then the cart contains the product with quantity 2
Scenario: Update cart quantity
  Given the cart contains a product
  When the user changes the quantity to 5
  Then the cart item quantity becomes 5
Scenario: Remove cart item
  Given the cart contains a product
  When the user removes the item
  Then the item is deleted from the cart
```

3.6 Checkout

```
Scenario: Successful checkout
  Given the cart contains products with sufficient stock
  When the user clicks "Place Order"
  Then an order is created
  And order items are created
  And product stock is reduced
  And the cart is cleared
  And the system returns HTTP 201
Scenario: Checkout with insufficient stock
  Given the cart contains a quantity greater than available stock
  When the user clicks "Place Order"
  Then the system returns HTTP 400
  And no order is created
```

3.7 Order Status Tracking

```
Scenario: Admin updates order status
  Given order 101 is in "Pending"
  When the admin changes the status to "Approved"
  Then the order status becomes "Approved"
  And updated_at is set
Scenario: Invalid status transition
  Given order 101 is in "Delivered"
  When the admin changes the status to "Pending"
  Then the system returns HTTP 400
```

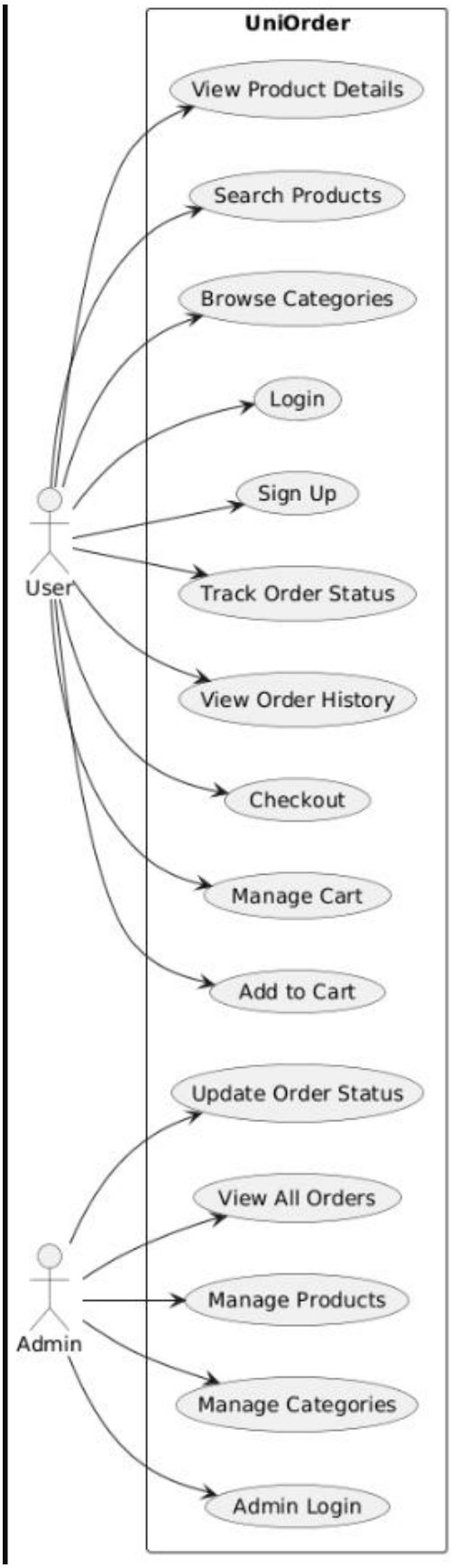
4. QA Refinement Audit Log

Original Requirement	Issue	Refined Requirement
Secure login	Not measurable	Passwords are hashed using bcrypt with a cost factor of 12 and JWT tokens expire after 24 hours.
Fast response	Vague	95% of API requests complete in 500 ms or less under normal load.
Scalable system	Vague	Supports at least 1,000 concurrent users and 10,000 products.
Reliable checkout	Vague	Checkout executes in a single database transaction with zero partial writes.
User-friendly search	Subjective	Search returns matching products with pagination metadata within 500 ms.
Real-time tracking	Ambiguous	Order status changes are visible to users within 2 seconds of update.

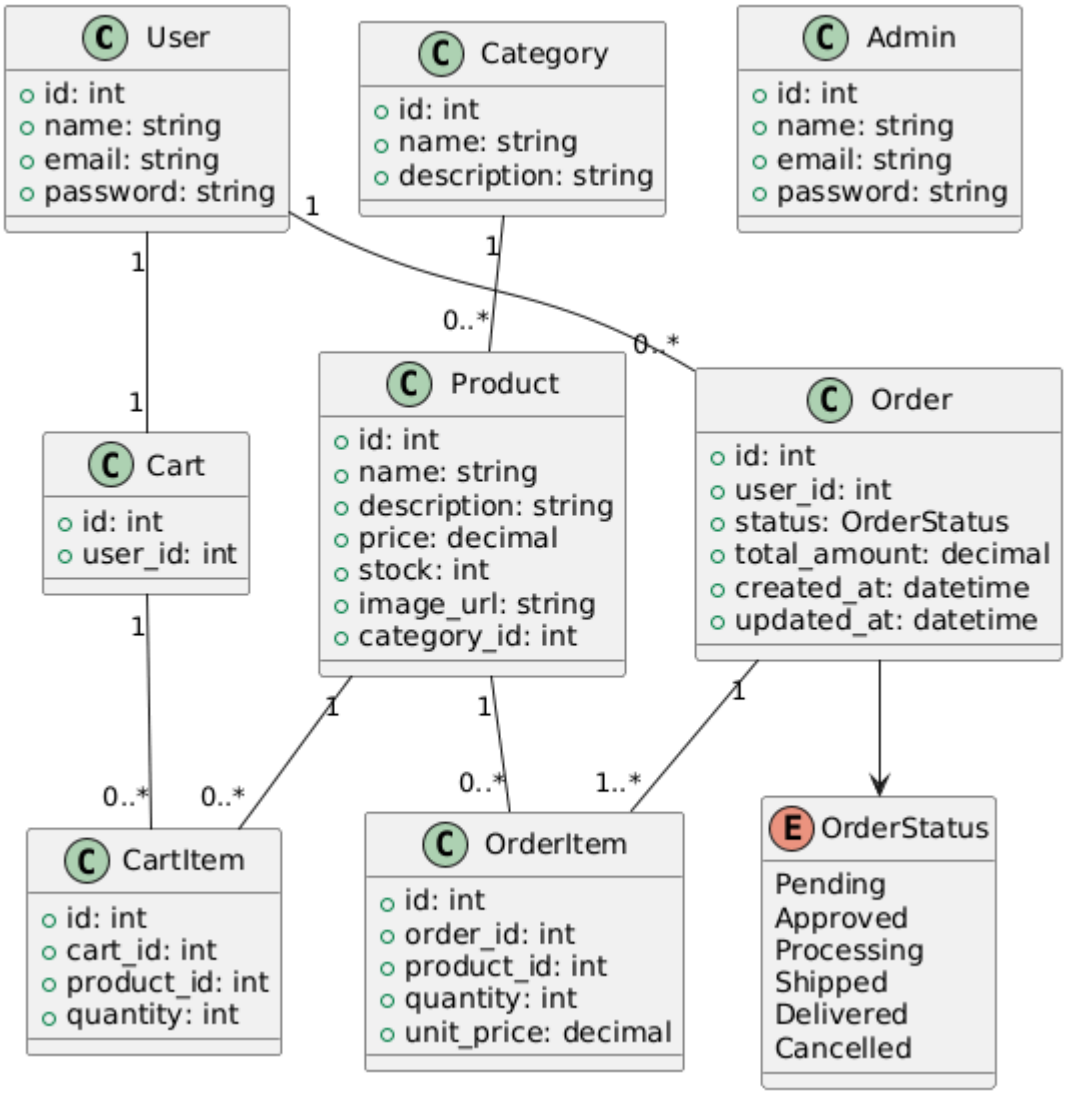
Original Requirement	Issue	Refined Requirement
Secure admin access	Vague	All admin endpoints require a valid JWT containing admin_id.
Accurate inventory	Vague	Stock values are updated atomically and never become negative.

5. UML Diagrams

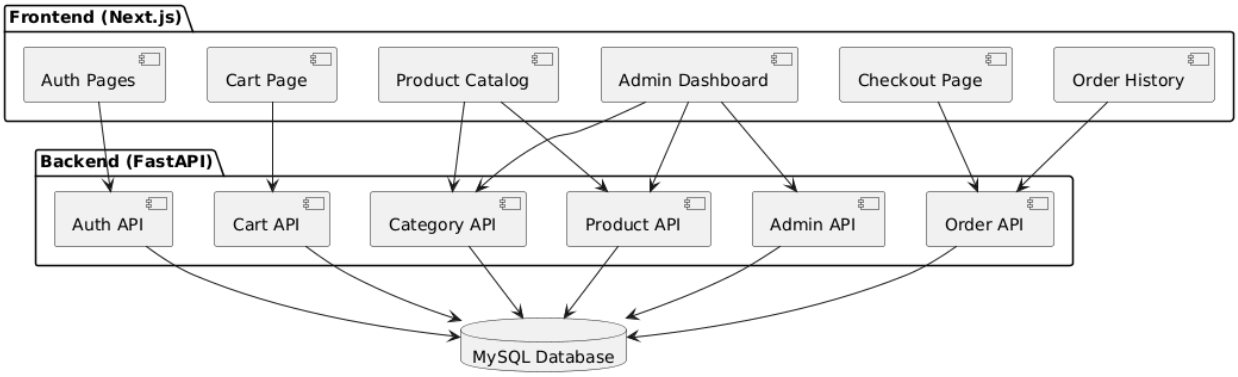
5.1 Use Case Diagram


UniOrder
View Product Details
Search Products
Browse Categories
Login
Sign Up
Track Order Status
View Order History
Checkout
Manage Cart
Add to Cart
Update Order Status
View All Orders
Manage Products
Manage Categories
Admin Login
User
Admin

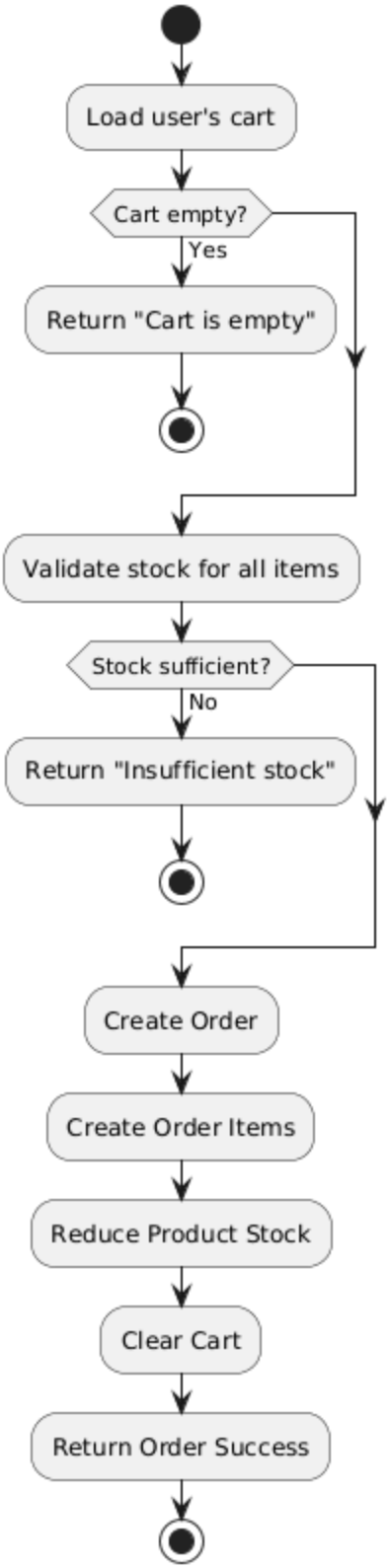
5.2 Class Diagram



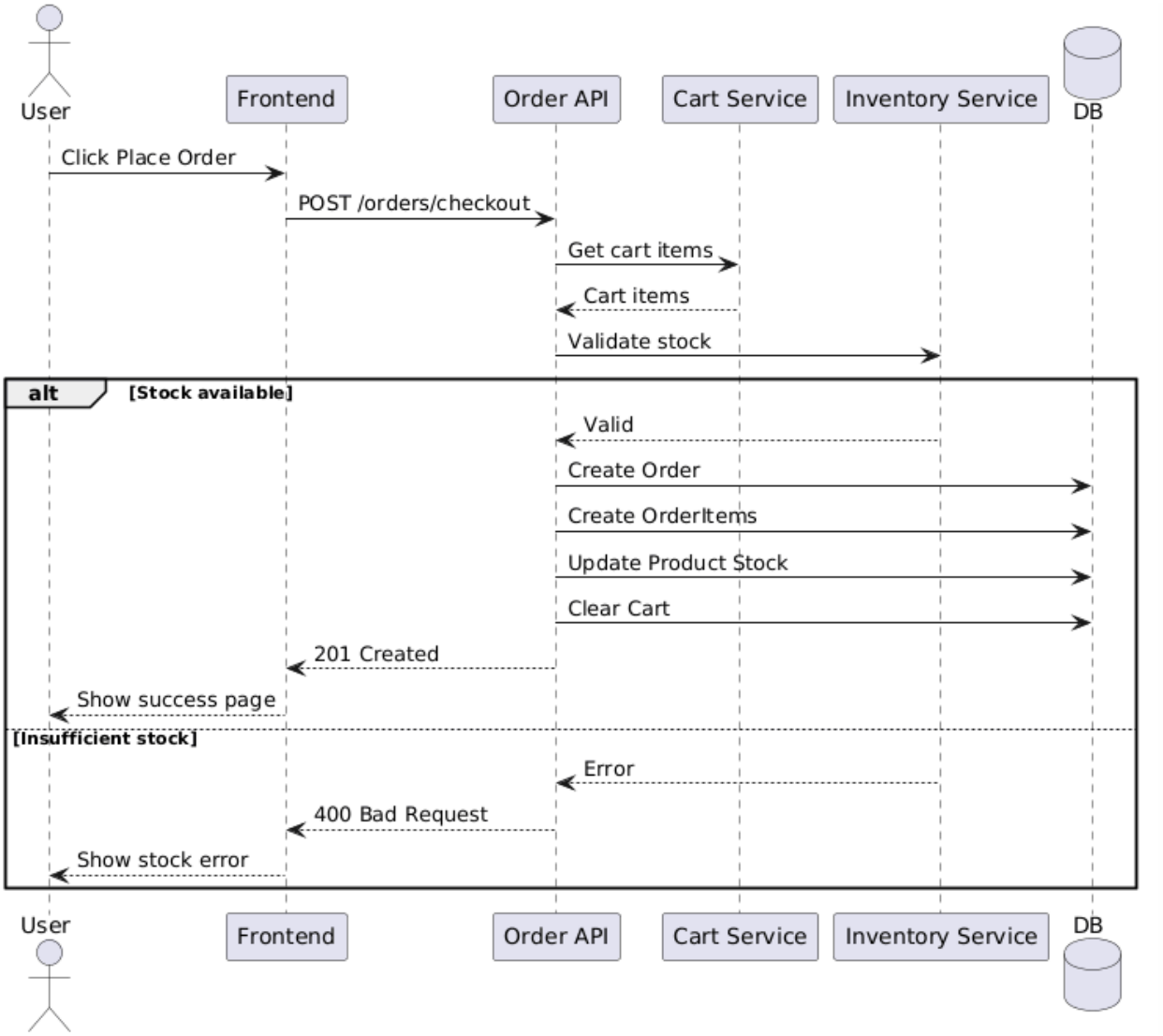
5.3 Component Diagram



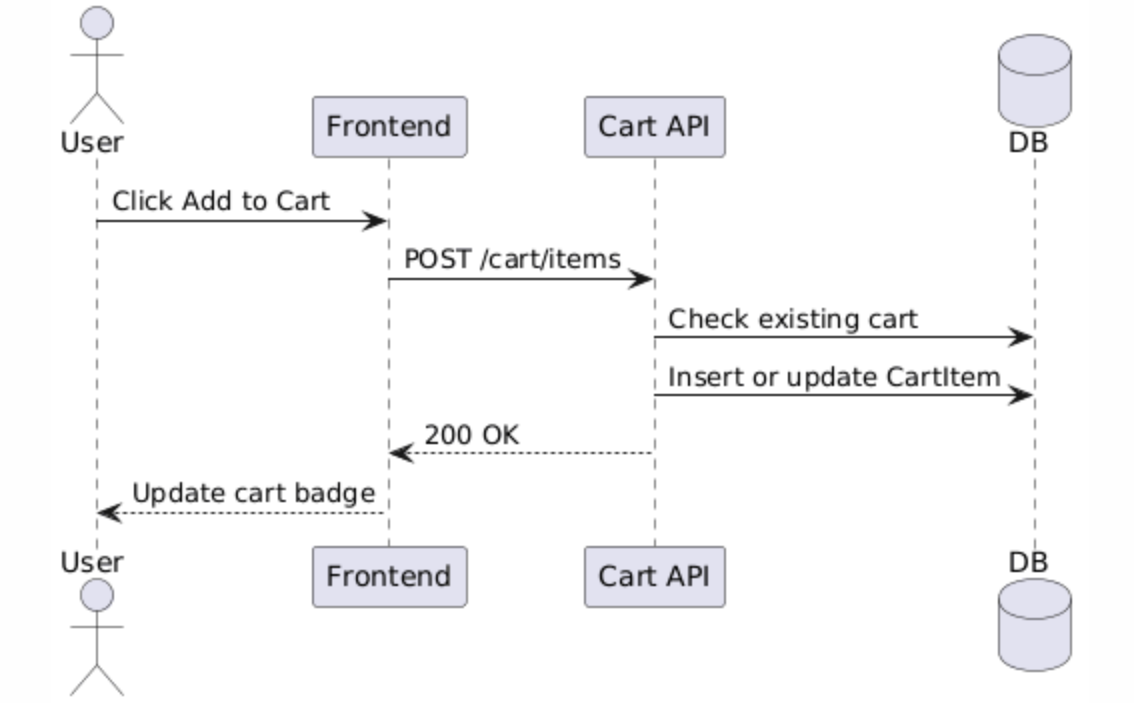
5.4 Activity Diagram – Checkout Workflow


Load user's cart
Cart empty?
Yes
Return "Cart is empty"
Validate stock for all items
Stock sufficient?
No
Return "Insufficient stock"
Create Order
Create Order Items
Reduce Product Stock
Clear Cart
Return Order Success

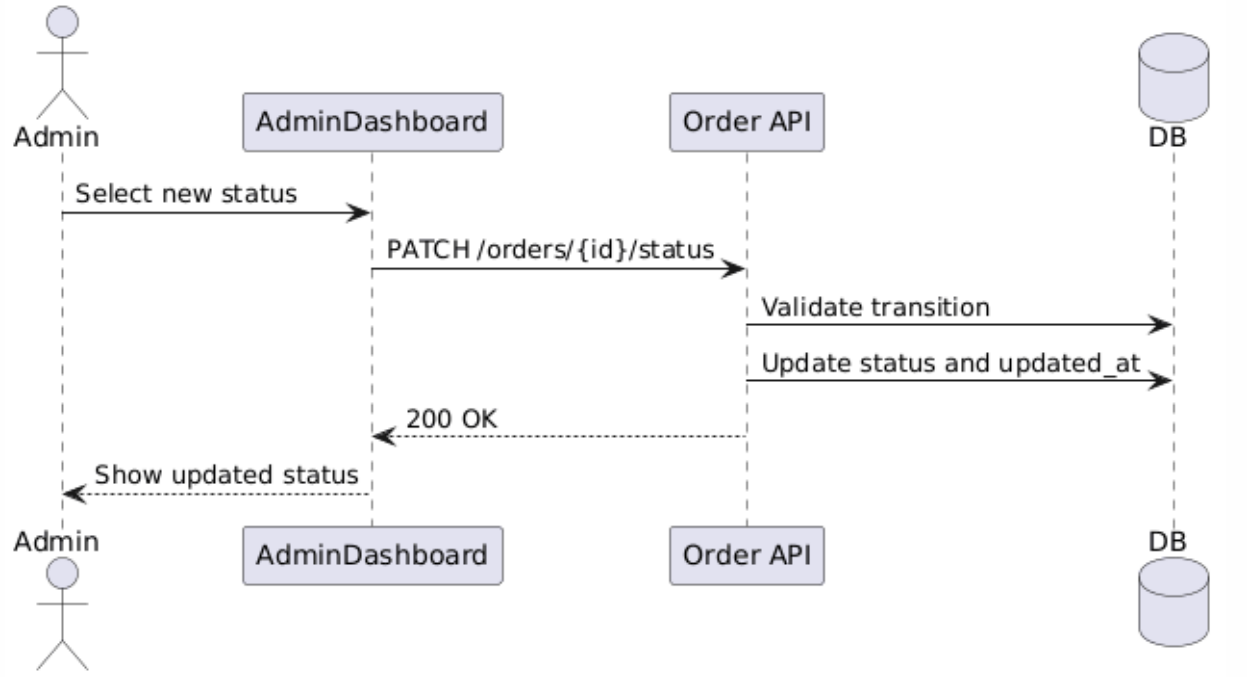
5.5 Sequence Diagram – Checkout (Happy and Failure Paths)



5.6 Sequence Diagram – Add Product to Cart



5.7 Sequence Diagram – Admin Updates Order Status



6. API Contracts & Information Hiding

Information Hiding Principle

- Database schema
- SQLAlchemy ORM models
- Password hashing logic
- JWT implementation
- Business rules

6.1 Authentication APIs

Method	Endpoint	Description	Auth
POST	`/auth/signup`	Register a new user	Public
POST	`/auth/login`	User login	Public
POST	`/admin/auth/login`	Admin login	Public

6.2 Category APIs

Method	Endpoint	Description	Auth
GET	`/categories/`	List all categories	Public
POST	`/categories/`	Create category	Admin
PUT	`/categories/{category_id}`	Update category	Admin
DELETE	`/categories/{category_id}`	Delete category	Admin

6.3 Product APIs

Method	Endpoint	Description	Auth
GET	`/products/`	List products with search/filter/pagination	Public

Method	Endpoint	Description	Auth
GET	`/products/{product_id}`	Product details	Public
POST	`/products/`	Create product	Admin
PUT	`/products/{product_id}`	Update product	Admin
DELETE	`/products/{product_id}`	Delete product	Admin

6.4 Cart APIs

Method	Endpoint	Description	Auth
GET	`/cart/`	Retrieve current cart	User
POST	`/cart/items`	Add item to cart	User
PUT	`/cart/items/{item_id}`	Update quantity	User
DELETE	`/cart/items/{item_id}`	Remove item	User

6.5 Order APIs

Method	Endpoint	Description	Auth
POST	`/orders/checkout`	Place order	User
GET	`/orders/history`	Order history	User
GET	`/orders/{order_id}`	Order details	User
PATCH	`/orders/{order_id}/status`	Update status	Admin
GET	`/admin/orders`	List all orders with filters	Admin